LLM Gateway — Error Handling, Request IDs & Timeouts

Interview preparation · V1 · standardized errors, X-Request-ID tracing, bounded timeouts

1. Error Handling

Predictable status codes and a single safe error shape.

Q1. How does your error handling work end to end?

MEDIUM

Answer: Business code raises typed exceptions (subclasses of GatewayError); it never builds HTTP responses. Centralized exception handlers turn any GatewayError into one consistent JSON envelope with the right status code, plus an X-Request-ID header. Unknown exceptions become a generic 500 with details logged server-side only.

Simple explanation: Raise a typed error; one place turns it into a clean HTTP response.

```
{ "error": { "code": "UNKNOWN_MODEL", "message": "..." } }
```

Interview tip: Stress 'routes don't build error JSON' — serialization is centralized.

Q2. What is your status-code and error-code mapping?

EASY

Answer: Each situation has an intentional status and a stable machine-readable code, so clients can branch on the code rather than parse messages.

Simple explanation: HTTP status says the category; the error code says the exact reason.

Situation	HTTP	Error code
Missing/invalid gateway key	401	UNAUTHORIZED
Unknown model alias	404	UNKNOWN_MODEL
Invalid request body	422	VALIDATION_ERROR
Upstream provider failed/timed out	502	PROVIDER_ERROR
Provider not configured	503	PROVIDER_UNAVAILABLE
Unexpected bug	500	INTERNAL_ERROR

Q3. How do you avoid leaking provider internals or secrets in errors?

HARD

Answer: Adapters translate raw provider failures into a generic ProviderError with a safe message, and log the real status/body server-side only. Unexpected exceptions return a generic 500 — no tracebacks, SQL, URLs, or keys reach the client. Validation errors are normalized and the raw input value is stripped so prompt content is never echoed back.

Simple explanation: Clients get a safe message; the real cause stays in server logs.

Project example: A test plants a fake secret in a provider error body and asserts it never appears in the client response (which is a generic 502).

Likely follow-ups:

- Why strip the 'input' from 422 errors? (Pydantic echoes the submitted value, which could be prompt text)

Interview tip: The 'generic to client, detailed in logs' split is the whole point.

Q4. Why distinguish 'known' errors from 'unexpected' ones?

MEDIUM

Answer: Known errors (bad key, unknown model, provider failure) are expected and safe to describe, with intentional status codes. Unexpected errors are bugs we can't describe safely, so they collapse to a generic 500. This keeps responses predictable and prevents accidental information disclosure.

Simple explanation: Expected problems get specific answers; surprises get a safe generic 500.

Project example: GatewayError subclasses carry status_code + error_code + message; the bare Exception handler owns the 500 path.

2. Request IDs

One id to trace a request across logs, DB, and response.

Q5. How do request IDs work and why do they matter?

MEDIUM

Answer: A middleware assigns exactly one request id per request (format req_ + uuid4 hex), stores it on request.state, and returns it as the X-Request-ID response header. The same id is used by usage logging and stored in the requests table, giving one correlation path from client to logs to database.

Simple explanation: One id per request, visible to the client and saved in the DB, so you can trace it.

```
Browser (X-Request-ID)  ==  requests.request_id  ==  server logs
```

Interview tip: Say: a user reporting an error can quote the X-Request-ID and you find the exact row.

Q6. Does the client get a request ID even on errors?

EASY

Answer: Yes. The id is attached to success and to all error responses (401, 404, 422, 502, 503, 500). The exception handlers read it from request.state and set the header, so even a failed request is traceable.

Simple explanation: Every response carries X-Request-ID, success or failure.

Project example: Can you support a client-supplied id? Yes — a valid X-Request-ID on the way in (safe format, length-limited) is reused; otherwise the server generates one.

Likely follow-ups:

- Why validate a client-provided id? (to prevent oversized/garbage header injection)

3. Timeouts

Never waiting forever on an upstream provider.

Q7. How do timeouts work when calling a provider?

MEDIUM

Answer: Every provider call uses httpx with a configurable timeout (LLM_TIMEOUT_SECONDS, default 30, validated to be > 0). If the provider is too slow, httpx raises a TimeoutException, the adapter catches it and raises a ProviderError with a safe 'request timed out' message, which becomes a 502 and is logged.

Simple explanation: If a provider takes too long, we stop waiting and return a clean 502.

Project example: This actually fired live: a slow Gemini call hit the 30s timeout, the client got a 502 PROVIDER_ERROR with X-Request-ID, and the DB row showed latency ~30000ms.

Interview tip: The real-life timeout story (and the fix below) makes this memorable.

Q8. Why no automatic retries or fallback on a timeout?

HARD

Answer: V1 keeps behavior predictable and avoids hidden cost/duplication: a timeout returns one clean error, full stop. Retries can amplify load on an already-struggling provider and double token spend; fallback across providers changes semantics. Both are deliberate V2 features, enabled by the clean adapter boundary but not in V1.

Simple explanation: One attempt, one honest error — no silent retries or provider-switching yet.

Project example: A test asserts exactly one provider call happens on timeout (no hidden retry). The practical fix for slow Gemini was choosing a faster default model, not adding retries.

Likely follow-ups:

- Where would retries live later? (around the adapter call, without touching routes or GatewayService)